

Every Map an Evolution, Every Room a Generation: Co-Evolution in a Procedurally-Generated Video Game

Erik Fredericks, frederer@gvsu.edu

GPTP, June 4th, 2026

<https://efredericks.github.io>

(Background gif: Binding of Isaac)

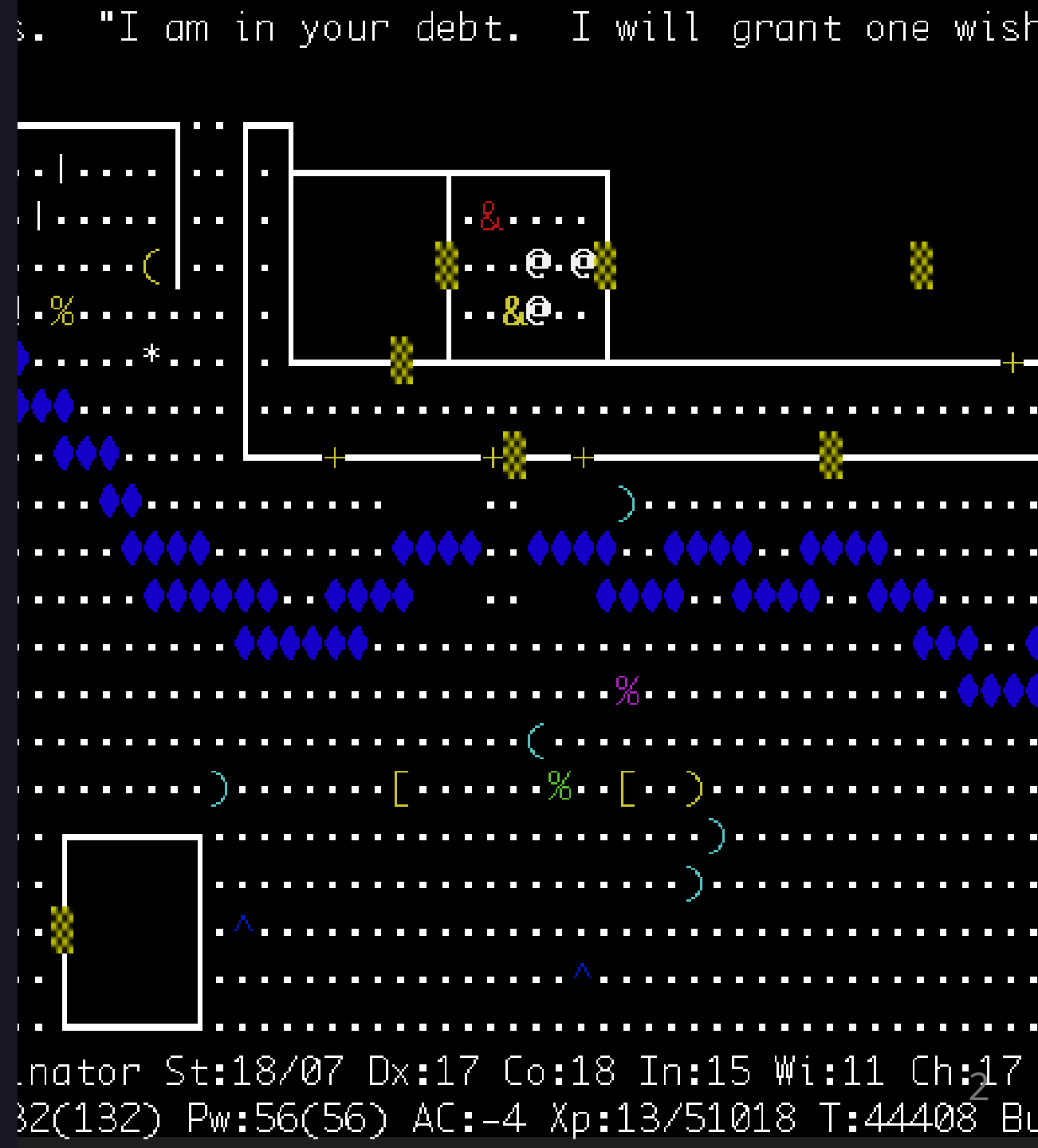


Premise

Create a fully-evolved two-dimensional roguelite game

**All aspects of procedural
content generation (PCG)
managed by genetic
programming (GP)**

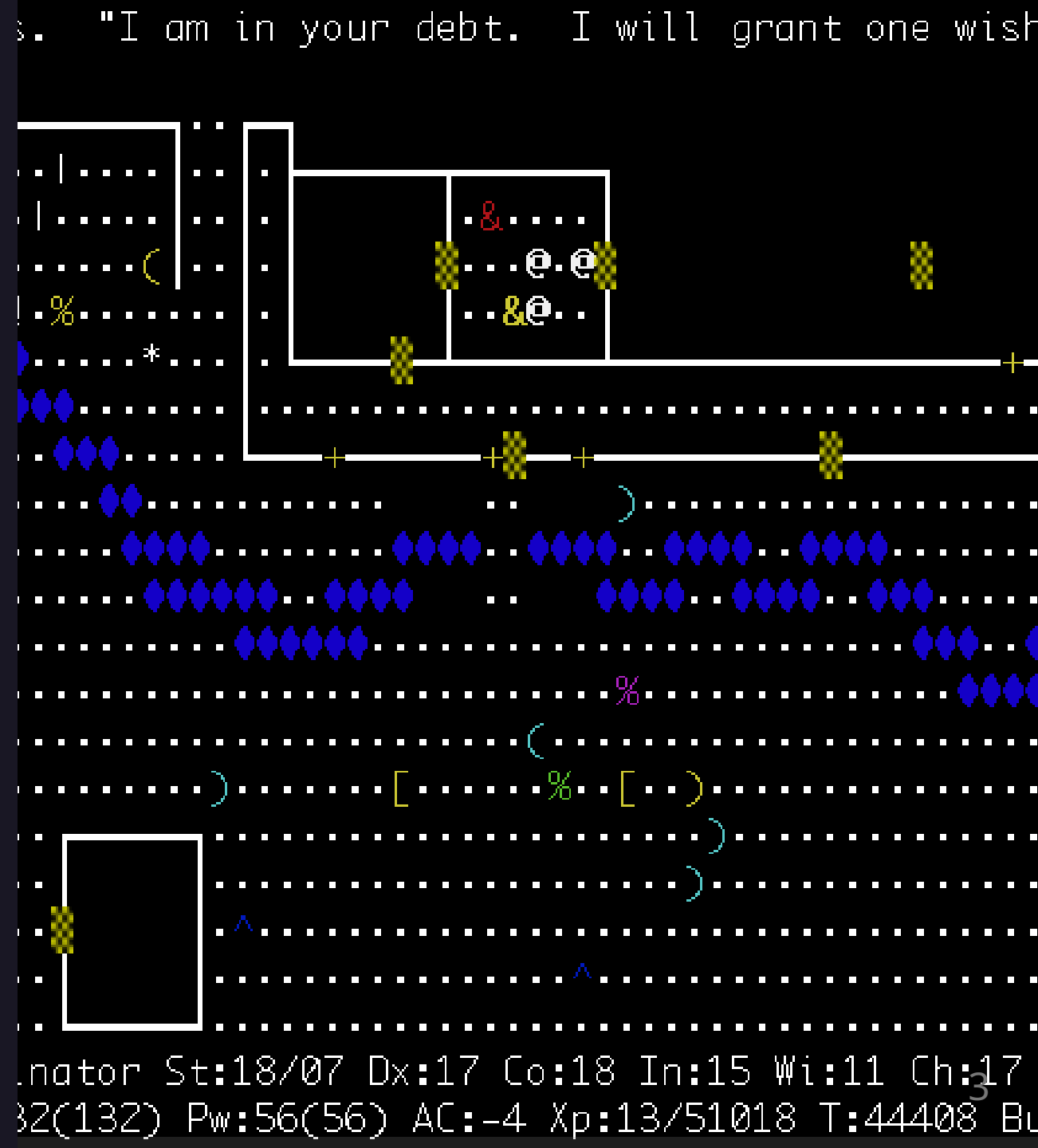
- Map creation
- Entity placement
- Enemy AI control



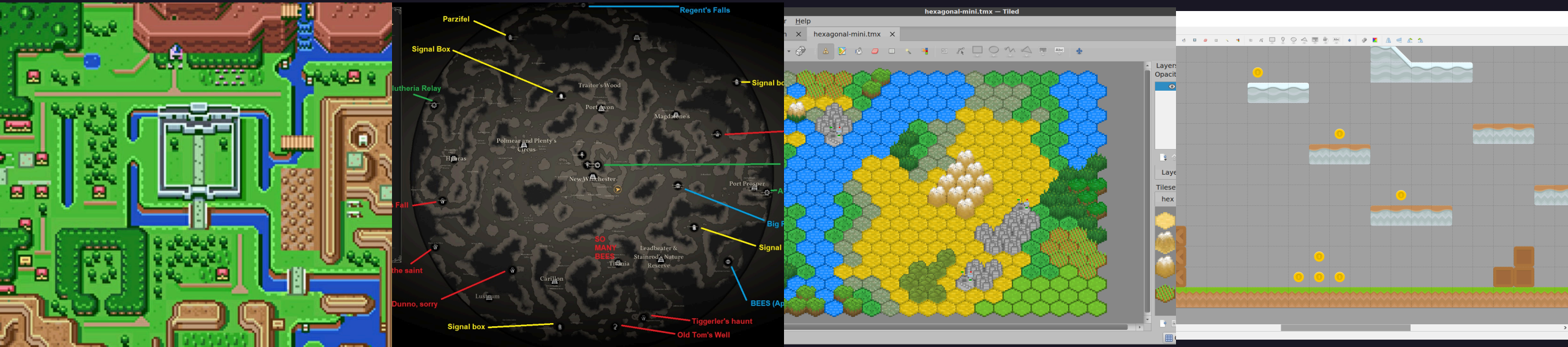
Status

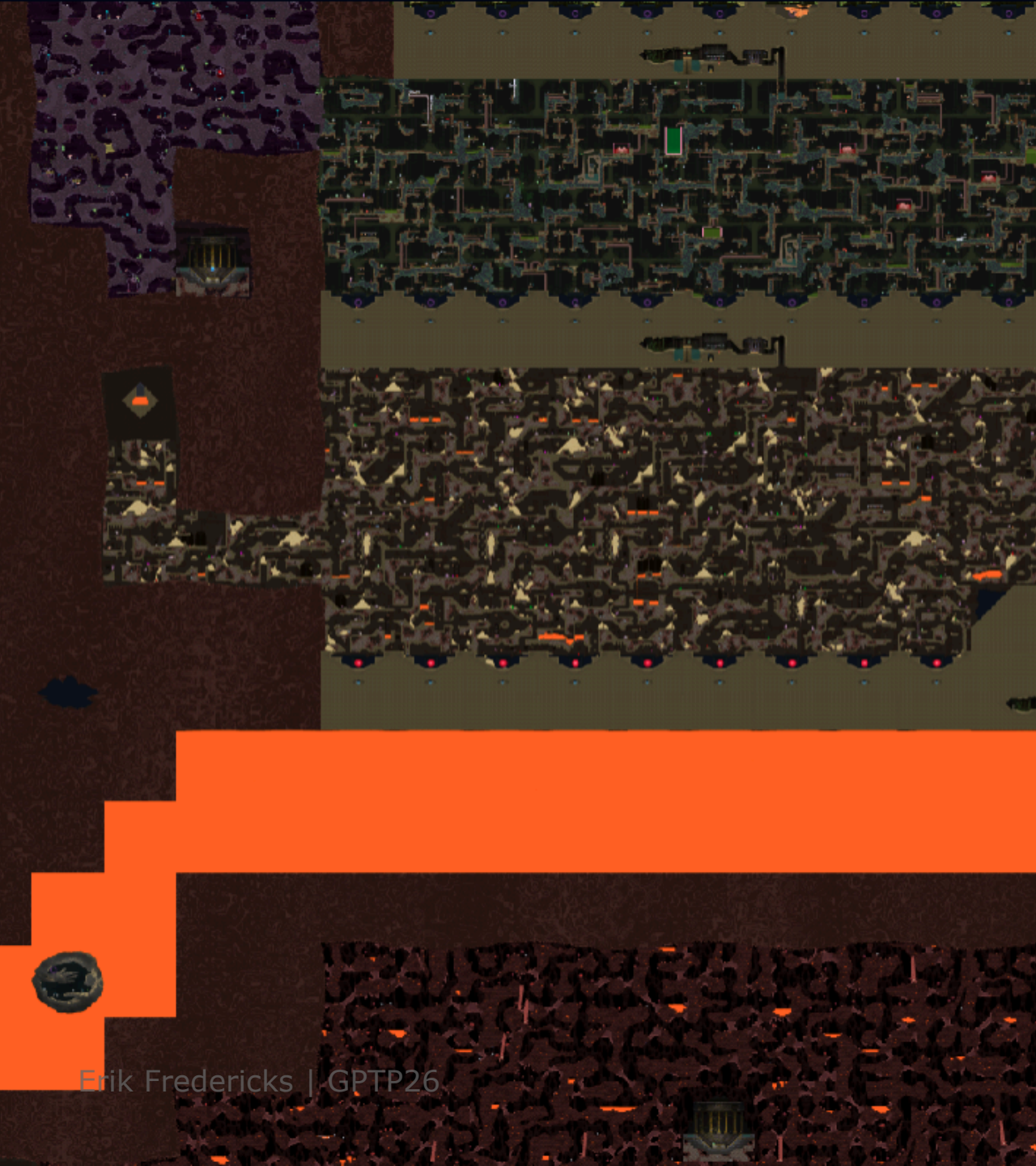
One GP active and one in progress

- Dungeon generation (functioning, optimization required)
- Enemy AI programming (hooks in place)
- GP co-evolution in progress



Making a game map





Erik Fredericks | GPTP26



x1?

Time: 00:32:32

Procedural Content Generation

What is PCG?

- Algorithmically-placing and generating content []

Why PCG?

- Save the developer/designer time and effort!
 - One would think...

Goal for GP?

- Optimize for variety and difficulty in game experience

Basic concept

Use math/algorithms to place content intelligently

- Dungeons, items, etc.

Noise functions:

- Calculate a noise value based on inputs and configurations
- Map that value to something useful

Optimizing PCG

Algorithms exist to be optimized

For example - Perlin noise []

For example (in p5):

```
let n = noise(x * 0.01, y * 0.01);
```

- `n = [0.0, 0.5]` -- `(x, y) -> water`
- `n = (0.5, 0.6]` -- `(x, y) -> beach`

There are other noise functions! Worley noise, Simplex noise, etc.

Noise function mappings:

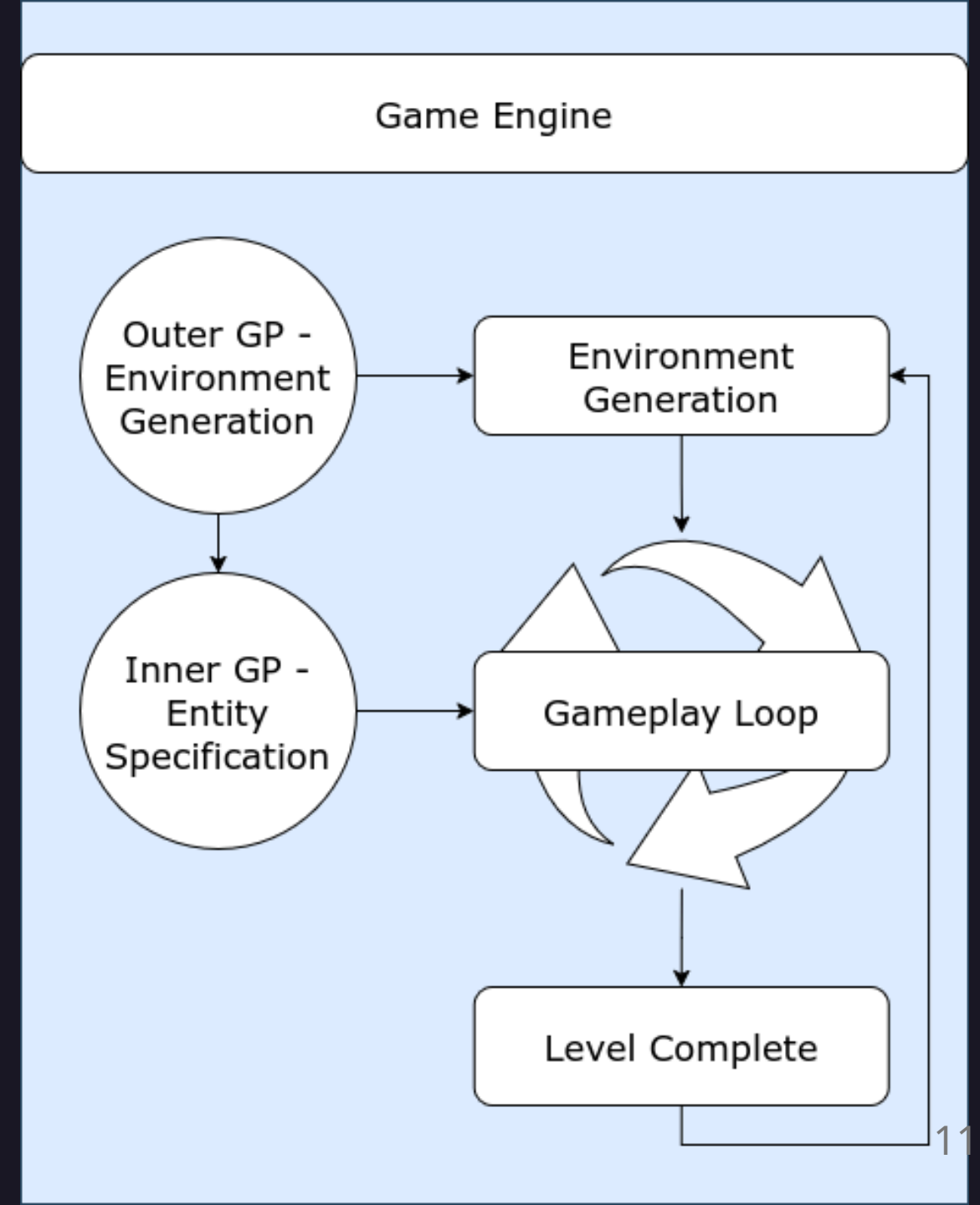
- Environment biome
- Map height
- Enemy spawns

Prior Art

There has been a *lot* done with PCG and evolutionary computation

- **Cooke *et al.* explored co-evolution for game generators []**
- **Stanley *et al.***

Workflow



Genome

1. **Individual room layouts**

- 2D grid of characters to represent obstacles, painful obstacles, items, and enemies

2. **Enemy programs** (tied to entities in room)

- List of actions to be executed in order

Fitness Functions

fitness = layout_score - difficulty_penalty

layout_score =

ratio of obstacles [0.1, 0.25]%

ratio of pain obstacles [0.03, 0.15]

ratio of open cells [0.5, 1.0]

ratio of obstacles in center [0, 3]

diversity of enemy program

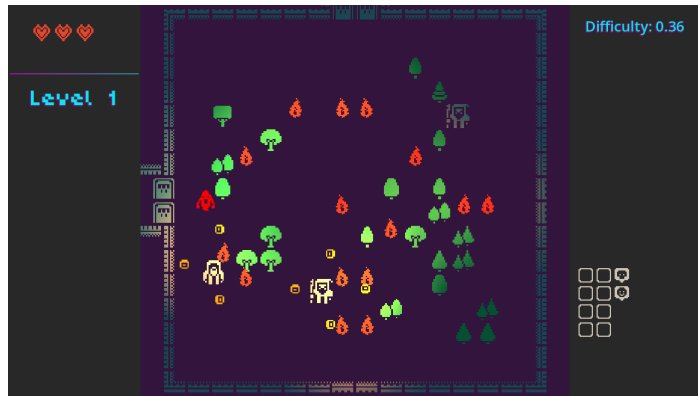
shape of room

maximize connectivity

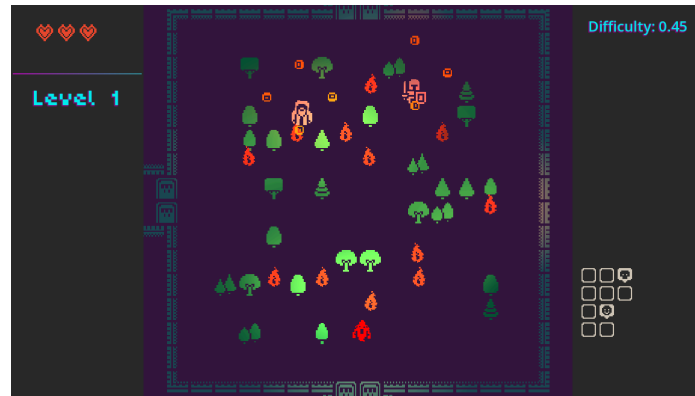
score chokepoints

score clustering

Sample Outputs (Room Generation)



Difficulty: 0.36



Difficulty: 0.45



Difficulty: 0.50

Sample Outputs (Enemy Program)

Enemy: WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, AIM, WAIT_20, FIRE_ONE

Enemy:

WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_AL
L, WAIT_40, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, AIM, WAIT_20,
FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8
, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20
, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_4
0, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40
, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_AL
L, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_
4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40

Next Steps

- **Additional fitness functions that reward room symmetry**
- **Log metrics for incorporating in future runs**
 - Not just all sandboxed runs
- **Full empirical evaluation**

Demo

The game is playable locally and in the browser

- Though, exiting just freezes the browser at the moment...
- Gameplay based on Zenva tutorial []

<https://efredericks.github.io/gp-roguelite/>

References

- [] Cook, M., Colton, S., & Gow, J. (2016). The angelina videogame design system—part i. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(2), 192-203.
- [] Cook, M., Colton, S., & Gow, J. (2016). The angelina videogame design system—part ii. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(3), 254-266.
- [] Togelius, J., Champandard, A. J., Lanzi, P. L., Mateas, M., Paiva, A., Preuss, M., & Stanley, K. O. (2013). Procedural content generation: Goals, challenges and actionable steps.
- [] Stanley, K. O., Bryant, B. D., & Miikkulainen, R. (2005). Real-time neuroevolution in the NERO video game. *IEEE transactions on evolutionary computation*, 9(6), 653-668.
- [] Pugh, J. K., Soros, L. B., Frota, R., Negy, K., & Stanley, K. O. (2017, September). Major evolutionary transitions in the voxelbuild virtual sandbox game. In *Artificial Life Conference Proceedings* (pp. 553-560).

Demo time

This time I made a pre-baked p5js template for you. Go to this link and

File -> Duplicate

- Make sure you login and save often

The sketch

- You should have a little ASCII happy face that you can move around with your arrow keys
- Your game map is represented as a **two-dimensional grid** where the first index is the row and the second index is the column
- There is also a basic camera that follows your player so that we can make big maps

The map

```
[['#', '#', '#', ..., '#'],  
 ['#', '.', '.', ..., '#'],  
 ['#', '.', 't', ..., '#'],  
 [...],  
 ['#', '#', '#', ..., '#']]
```

`game_map[2][1]` returns a ???

- `.`

By default, the map puts walls on the outside and nothing on the inside - your job is to fill it with things

Try hitting ~ and see what happens!

Valid things:

Character	Walkable?	Represents
#	No	Stone wall
t	No	Tree
w	No	Water
	Yes	Empty space
.	Yes	Pebbles
g	Yes	Grass



Try setting some random cells to these characters!

- such as, `game_map[2][3] = 'g';` in `setup()` AFTER the game map is created

First, let's randomize things

After the call to `setupGameMap()` (which again, just gives us an empty grid with borders)

```
for (let r = 1; r < num_rows - 1; r++) {  
  for (let c = 1; c < num_cols - 1; c++) {  
    let r = random();  
    let ch = ' ';  
    if (r > 0.8) ch = '#';  
  
    game_map[r][c] = ch;  
  }  
}
```

Now let's give it a bit of detail

First, comment out what you did inside that double loop.

```
const zoom = 0.01;
for (let r = 1; r < num_rows - 1; r++) {
  for (let c = 1; c < num_cols - 1; c++) {
    let ch = '';
    let n = noise(c * zoom, r * zoom);

    // fill in with code from next slide

    game_map[r][c] = ch;
  }
}
```

The noise bit

```
if (n <= 0.5) ch = 'w';  
else if (n <= 0.6) ch = 'b';  
else if (n <= 0.9) ch = 't';  
else {  
    let r = random();  
    if (r > 0.8) ch = '.';  
    else ch = ' ';  
}
```

The finesse bit

Now this is the tricky part - you need to play with the `zoom` value and the `noiseDetail` (at the top) values to get exactly the output you want!

Try varying the:

- `zoom` into the noise distribution (try `0.1`, `0.001`, etc.)
- <https://p5js.org/reference/p5/noiseDetail/>
 - The number of octaves (first parameter in `noiseDetail`) - `[1, 16]` usually show interesting values
 - The falloff amount (second parameter in `noiseDetail`) - `[0.0, 1.0]`

Other ways to do it!

Use a different algorithm!

- Rectangular room placement
- Cellular automata
- Wave function collapse
- ...

